

实 验 报 告

实验名称: 实验二 感知器模型实现并进行分类

课程名称: 认知科学与类脑计算

实验地点: K2 211

实验日期: 2025/9/26

班 级: 23 级智能班

姓 名: 宋浩宇

学 号: 202300130183

一 实验目的：

加深对感知器模型的理解，能够使用感知器模型解决简单的分类问题

二 实验环境：

主系统：Windows 11 家庭中文版 23H2 ; ArchLinux

编辑器：Visual Studio Code ; PyCharm 2025.1.2 ; neovim 0.11

Python 解释器：Python 3.9

Pytorch 版本：2.8.0

CUDA 版本：12.8.97

硬件环境：

CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz

内存：32.0 GB (31.6 GB 可用)

磁盘驱动器：NVMe WD_BLACKSN850X2000GB

显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU

CUDA 核心数：7424

三 实验内容：

根据感知器的相关知识，使用 Python 语言实现一个简单的感知器模型，该模型能够实现简单的二分类任务（与或非运算）。

四 实验步骤：

1. 实验程序：

```
import numpy as np
class dataset:
    def __init__(self, data, label):
        self.data = data
        self.label = label
and_dataset = dataset(np.array([[0,0],[0,1],[1,0],[1,1]]),
np.array([0,0,0,1]))
or_dataset = dataset(np.array([[0,0],[0,1],[1,0],[1,1]]),
np.array([0,1,1,1]))
not_dataset = dataset(np.array([[0],[1]]), np.array([1,0]))
xor_dataset = dataset(np.array([[0,0],[0,1],[1,0],[1,1]]),
np.array([0,1,1,0]))
learning_rate = 0.1
epochs = 1000
def sigmoid(x):
    return 1/(1+np.exp(-x))
class Perceptron:
```

```

    def __init__(self, dataset, learning_rate=learning_rate,
epochs=epochs):
        self.dataset = dataset
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = np.random.rand(len(dataset.data[0]))
        self.bias = np.random.rand(1)
    def forward(self, x):
        z = np.dot(x, self.weights)
        return sigmoid(z + self.bias)
    def train(self):
        for epoch in range(self.epochs):
            for x, y in zip(self.dataset.data, self.dataset.label):
                y_pred = self.forward(x)
                error = y - y_pred
                self.weights += self.learning_rate * error * x
                self.bias += self.learning_rate * error
    def predict(self, x):
        y_pred = self.forward(x)
        if y_pred >= 0.5:
            return 1
        else:
            return 0
and_perceptron = Perceptron(and_dataset)
or_perceptron = Perceptron(or_dataset)
xor_perceptron = Perceptron(xor_dataset)
not_perceptron = Perceptron(not_dataset)
and_perceptron.train()
or_perceptron.train()
xor_perceptron.train()
not_perceptron.train()
print("AND Perceptron")
print("{} -> {}".format(and_dataset.data[0],
and_perceptron.predict(and_dataset.data[0])))
print("{} -> {}".format(and_dataset.data[1],
and_perceptron.predict(and_dataset.data[1])))
print("{} -> {}".format(and_dataset.data[2],
and_perceptron.predict(and_dataset.data[2])))
print("{} -> {}".format(and_dataset.data[3],
and_perceptron.predict(and_dataset.data[3])))
print("OR Perceptron")
print("{} -> {}".format(or_dataset.data[0],
or_perceptron.predict(or_dataset.data[0])))

```

```

print("{} -> {}".format(or_dataset.data[1],
or_perceptron.predict(or_dataset.data[1])))
print("{} -> {}".format(or_dataset.data[2],
or_perceptron.predict(or_dataset.data[2])))
print("{} -> {}".format(or_dataset.data[3],
or_perceptron.predict(or_dataset.data[3])))
print("XOR Perceptron")
print("{} -> {}".format(xor_dataset.data[0],
xor_perceptron.predict(xor_dataset.data[0])))
print("{} -> {}".format(xor_dataset.data[1],
xor_perceptron.predict(xor_dataset.data[1])))
print("{} -> {}".format(xor_dataset.data[2],
xor_perceptron.predict(xor_dataset.data[2])))
print("{} -> {}".format(xor_dataset.data[3],
xor_perceptron.predict(xor_dataset.data[3])))
print("NOT Perceptron")
print("{} -> {}".format(not_dataset.data[0],
not_perceptron.predict(not_dataset.data[0])))
print("{} -> {}".format(not_dataset.data[1],
not_perceptron.predict(not_dataset.data[1])))
print("End")

```

2. 程序实验结果:

AND Perceptron

[0 0] -> 0

[0 1] -> 0

[1 0] -> 0

[1 1] -> 1

OR Perceptron

[0 0] -> 0

[0 1] -> 1

[1 0] -> 1

[1 1] -> 1

XOR Perceptron

[0 0] -> 1

[0 1] -> 0

[1 0] -> 0

[1 1] -> 0

NOT Perceptron

[0] -> 1

[1] -> 0

End

3. 分析和改进:

从结果上来看，与或非三种逻辑运算，都是可以正确预测的，但是我顺便测了一个异或运算，得到的模型并不能正确预测，原因的话我猜想了一下，大概是因为异或运算实际实现，如果纯靠与或非那至少需要六个逻辑门组合，使得这个运算信息变化跨度太大了（？）实验要求的三个模型准确率已经到 100%了，我也想不出怎么改进了，或许还可以试试与非或非。

五 实验小结：

从结果上来看，感知机模型可以正确应对与或非这类简单逻辑运算的情况，甚至正确率能到 100%，不过异或运算还是不行。